

# Exercise 3: Give Claude Your Team's Decisions

**Session:** 3 — Context Engineering

**Duration:** 45 minutes, plus a 5-minute closing round

**Project:** The same BookStore API.

## Goal

---

Your team made three decisions last sprint. Claude was not in the meeting. In this exercise you:

1. Run a task without the decisions, and save what Claude builds.
2. Put each decision where Claude reads it: two rule files and `CLAUDE.local.md`.
3. Run the same task again, and let the grader compare the two results.

## Start here

---

- Open a terminal in the `bookstore` folder and start `claude`.
- Run `go test ./...` and `git status`. A test fails, or `git status` lists changes? Press Shift+Tab until the screen says auto mode, then run `/catch-up 2`. It commits your session 2 work, which the reset in task 1 would otherwise remove.
- Training mode is on. Never edit `CLAUDE.md`. Stuck? Type *just tell me*.
- The trainer tells you when to start task 3.

## Your team's decisions

---

From the notes of last sprint's retrospective:

1. **Test kit.** "We built a test kit: `internal/handler/testkit_test.go`. From now on, new handler tests use it. We do not rewrite the old tests."
2. **Bad input.** "Bad input must give a 400, not a 500. New endpoints do this. We fix the old endpoints later, in a separate change."
3. **Small changes.** "A change touches only what the task asks for. No tidying up on the side."

Claude cannot read these notes. Nothing in the code says them either: the old tests and the old handlers still show the old way.

# Tasks

---

## 1. Run the task without the decisions

This task produces `session3-before.diff`: what Claude builds when it knows only the code.

1. **Predict.** For each decision, write *yes* or *no*: will Claude follow it without being told? A wrong prediction is fine. No prediction is the only failure.
2. **Run the task in a fresh session.** Type `/exit` and start `claude` again. A fresh session always means this: close Claude and start it again, never `/clear`. Then paste this exactly. The tag at the start switches training mode off for this one prompt:

```
[Exercise 3 experiment – execute directly, no leading questions.] Add a DELETE
/api/reviews/{id} endpoint to the BookStore API, with tests.
```

3. **Run the tests.** When Claude is done, run `go test ./...`. Write down whether all tests pass.
4. **Save the result.** Run `/save-changes before`. It saves all changes to `session3-before.diff` and resets the project. Then type `/exit`.

**Done when:** your three predictions are written down, and `session3-before.diff` exists.

## 2. Put the decisions where Claude reads them

This task produces `.claude/rules/testing.md`, `.claude/rules/handlers.md` and `CLAUDE.local.md`. Each decision goes where it is needed, and only there (*Rule Discovery: With or Without Paths*).

1. **Decision 1: let Claude create the rule.** Start `claude` and paste this whole block:

[Exercise 3 experiment – execute directly, no leading questions.] Create `.claude/rules/testing.md` with exactly this content, nothing added or changed:

```
---
description: How we write new handler tests
paths:
  - "internal/handler/*_test.go"
---
New handler tests use the test kit in internal/handler/testkit_test.go.
Read that file before you write a test. Use this shape: one Test<Method><Resource>API
function per endpoint, one t.Run per case.
A new test never calls newTestMux or httptest.NewRequest directly, even when the tests
around it do.
Never convert an existing test to the kit, also not in a file you edit.
```

Look at the parts of this rule. `paths:` loads the rule when Claude opens a test file. The kit line points to the kit, so the details stay in one place (*Progressive Disclosure*). The "even when" line says what to do when the code shows the old way. The last line is the limit: new tests only.

2. **Decision 2: write the rule yourself.** Open your editor and create `.claude/rules/handlers.md`. Copy the shape of `testing.md`:

- A `paths:` list with `"internal/handler/**/*.go"`, the handler files.
- One sentence on what new endpoints do with bad input.
- One sentence on what stays the same, and where.

A rule must be checkable: someone who reads a diff can say "this breaks the rule". *"Handle errors well"* is not checkable. *"Return 404 when the book does not exist"* is.

3. **Decision 3, and a line that points to the rules.** Create `CLAUDE.local.md` in the `bookstore` folder, in your editor. Claude reads this file at the start of every session. Write two lines:

- First, copy this line exactly: *"Before you write or edit a file under internal/handler/, read the matching file in .claude/rules/."*
- Then decision 3, as one checkable rule. It applies to every file, so it gets no `paths:`. Name what a change may touch, so a diff can show a violation.

Why the first line: `paths:` loads a rule only when Claude opens a matching file with its own file tools. When Claude creates a new file, or edits through a shell command or a script, the rule may not load. The line in `CLAUDE.local.md` is always loaded, so it closes that gap.

4. **Optional: ask the coach, one round.** Only if the trainer has not called task 3 yet. Run `/context-coach 2` (2 is the task number), then paste your `handlers.md` and your decision 3 line as your next message. Fix the one thing it names, then go on.

**Done when:** the three files exist. `handlers.md` has a `paths:` line and says which endpoints the rule does not cover. `CLAUDE.local.md` has the line that points to the rules. `git status` does not list `CLAUDE.md`.

### 3. Run the same task with the decisions

This task produces `session3-after.diff`.

1. **Start a fresh session.** Type `/exit` and start `claude` again. A fresh session reads your new files from the start.
2. **Paste the prompt from task 1 again**, tag included. Watch which files Claude opens before it writes a test. Does it open the test kit?
3. **Run the tests.** When Claude is done, run `go test ./...`. Write down whether all tests pass.
4. **Save the result.** Run `/save-changes after`. Then type `/exit`.

**Done when:** `session3-after.diff` exists.

### 4. Closing: let the grader compare

Start `claude` and run `/verify-exercise 3`. It first asks which decision you think Claude still broke in the second run. Answer in one line; *none* is a fine answer. Then it compares the two diffs, decision by decision: *followed*, *broken*, or *not tested* when the task gave no chance to break it. For each result it shows the lines that decided it.

Write down three lines:

1. Your predictions from task 1, next to what the first run did.
2. The decision that changed most between the two runs.
3. One sentence from your rules that you would now write differently.

**Done when:** three lines are written down. Bring them to the closing round.

## Bonus (only if time remains)

---

**Same rules, a new task.** Start a fresh session and paste:

```
[Exercise 3 experiment – execute directly, no leading questions.] Add a GET
/api/authors/{id}/books endpoint to the BookStore API, with tests.
```

Run `/save-changes bonus`, then `/verify-exercise 3` in a fresh session. You wrote the rules once. Do they hold for a task you did not plan for?

**Change an old test.** Start a fresh session and paste:

```
[Exercise 3 experiment – execute directly, no leading questions.] In the existing review
tests, make the test for listing reviews also check that the first review has rating 5.
```

Run `git diff`. Did Claude change only that test, or did it convert it to the kit? Then run `/save-changes probe` to reset the project.

## Back at work: try this on your own project

---

Every review comment you write twice is a rule you have not written yet. Pick one comment you keep repeating, and write it as a rule with a `paths:` line. In a real team, `.claude/rules/` is committed, so every teammate gets the same rules. In this course, git ignores it.

## Appendix: `/save-changes` by hand

---

If the command stops, it says why. Or run these from the project folder, one per line:

```
git add -A .
git diff --cached --output=session3-before.diff
git reset -q .
git checkout -- .
git clean -fd .
```

Replace `before` with `after`, `bonus` or `probe`. The last command deletes files Claude created that you never committed. Your rule files, `CLAUDE.local.md` and the saved diffs are ignored by git and stay.

## Closing round

---

The trainer asks the room. Have these answers ready:

- Which decision did Claude follow without being told? Did your prediction hold?
- Why does decision 1 go in a rule file with `paths:` , and decision 3 in `CLAUDE.local.md` ? Why does `CLAUDE.local.md` also point to the rules?
- Which words in your rule for decision 2 make it checkable?
- One thing you would tell someone who skipped this session.